

Secure Defense Asset Management and Lifecycle Tracking System

DBMS Project Report

Subject: Database Management System

Project Title: Secure Defense Asset Management and Lifecycle Tracking System

Technologies Used: Oracle SQL, PL/SQL, Oracle Live SQL, HTML/CSS/JavaScript

Team Members:

1. Harsh Rakesh Arora - 1024031161
 2. Srijan Srivastava - 1024030903
 3. Manish Garg - 1024030911
-

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. System Design
6. ER Diagram
7. Relational Schema
8. Normalization
9. SQL Implementation
10. PL/SQL Implementation
11. Transaction Management & Concurrency
12. Frontend Dashboard
13. Challenges Faced
14. Future Scope
15. Conclusion

1. Introduction

Defense organizations manage large inventories of high-value assets — including weapons, ammunition, and tactical equipment — that must be tracked throughout their lifecycle from acquisition and allocation to maintenance and decommissioning. Such assets require strict accountability, accurate records, and timely maintenance to ensure operational readiness and institutional transparency.

This project implements a relational database system for managing defense asset inventory, custody chains, maintenance schedules, ammunition compatibility, and audit trails. The system is built using Oracle SQL and PL/SQL, with business logic implemented directly in the database layer through stored procedures, functions, triggers, and cursors.

1.1 Why DBMS over File-Based Systems

A file-based approach for defense asset tracking would lead to data redundancy, inconsistencies during transfers, and weak accountability. A DBMS-based approach provides:

- **ACID-compliant transactions** ensuring atomic asset transfers
- **Referential integrity** through primary and foreign keys
- **Role-based access** via database views
- **Automated workflows** through triggers and procedures
- **Efficient querying** and reporting for compliance and decision-making

1.2 SDG-16 Alignment

The project aligns with **UN Sustainable Development Goal 16: Peace, Justice and Strong Institutions** by supporting responsible governance, accountability, and safe management of critical public-security infrastructure.

2. Problem Statement

2.1 Real-World Problem

Organizations handling high-accountability defense assets face challenges in maintaining accurate custody records, maintenance history, and asset availability across multiple units. Manual systems often result in missing records, delayed maintenance, and difficulty generating compliance reports.

2.2 Limitations of Existing/Manual Systems

Limitation	Impact
Spreadsheet/paper-based tracking	Prone to human error and data loss
No automated maintenance alerts	Delayed servicing and reduced operational readiness
Limited historical querying	Difficulty tracing custody chains and usage history
No transaction integrity	Inconsistent records during custody transfers
Weak access control	Unauthorized access and data modifications

2.3 Proposed Solution

A centralized relational database acting as the single source of truth for asset inventory, custody history, maintenance records, ammunition compatibility, and audit logging with role-based data access implemented through SQL views and automated workflows through PL/SQL.

3. Objectives

- ER Modeling:** Design an Entity-Relationship model representing assets, units, custodians, maintenance, and compatibility data.
 - Normalization:** Convert the ER model into a normalized relational schema up to 3NF/BCNF.
 - SQL Implementation:** Implement the database using SQL with joins, subqueries, aggregate queries, and views.
 - PL/SQL Automation:** Develop stored procedures, functions, triggers, and cursors for workflow automation.
 - Data Consistency:** Ensure data consistency using constraints, transactions, and concurrency control.
 - ACID Compliance:** Demonstrate ACID-compliant custody transfer and lifecycle tracking workflows.
-

4. Scope of the Project

4.1 In Scope

Module	Description
Weapon Registration & Inventory	Handles weapon registration, specifications, operational status, and location tracking
Custody Chain Management	Maintains transfer history with authorization and accountability validation
Maintenance Scheduling	Tracks servicing schedules and generates automated maintenance alerts
Ammunition Compatibility	Stores weapon–ammunition compatibility data with effectiveness ratings
Audit Logging	Records all critical database operations for accountability and traceability
Role-Based Access Control	Restricts data visibility and operations based on user roles and permissions

4.2 Out of Scope

- Real-time IoT integration or physical access control systems
- Financial procurement workflows
- External inter-agency data exchange
- User authentication (demonstrated through views only)

4.3 Types of Users

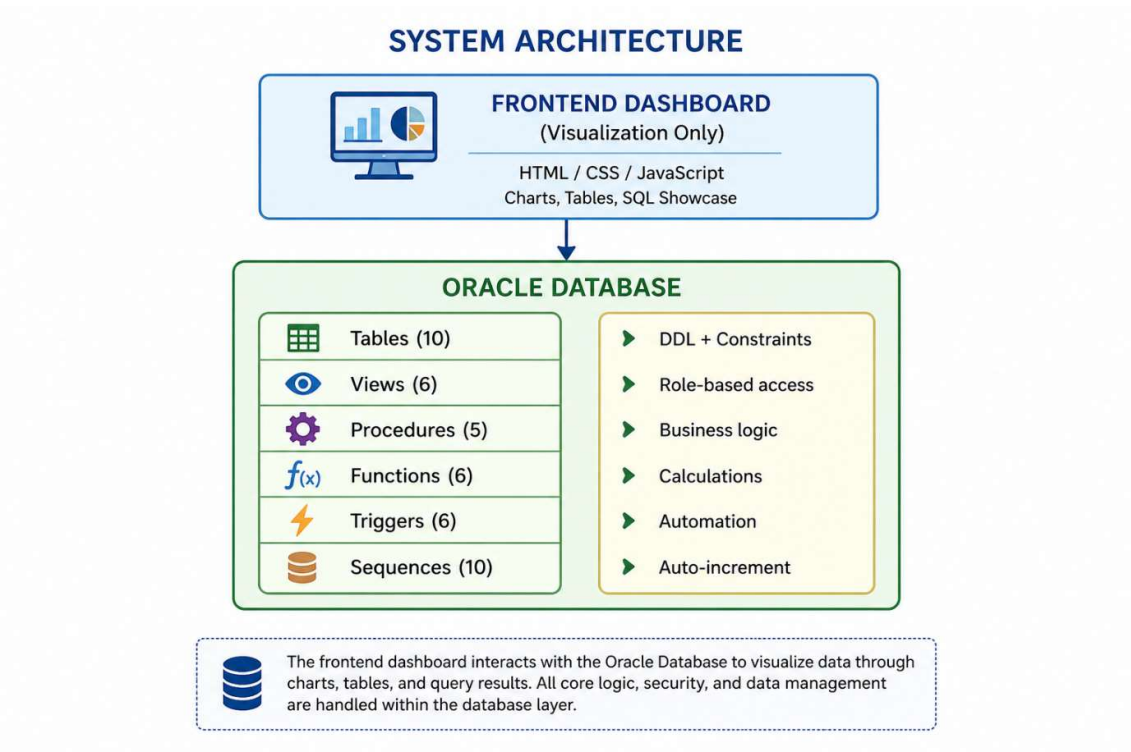
User Role	Access Level
Armory Officer	Full access to inventory management, custody transfers, and maintenance operations
Unit Commander	Read access to assigned assets and ability to request transfers
Maintenance Technician	Permission to update maintenance schedules and service records

User Role	Access Level
Auditor	Read-only access to audit logs, compliance records, and reports
Database Administrator	Responsible for schema management, backup, recovery, and database maintenance

5. System Design

5.1 Architecture

The system follows a **database-centric architecture** where all business logic resides in the Oracle database layer. There is no middle-tier application server; constraints enforce valid data states, while triggers and procedures automate workflows.

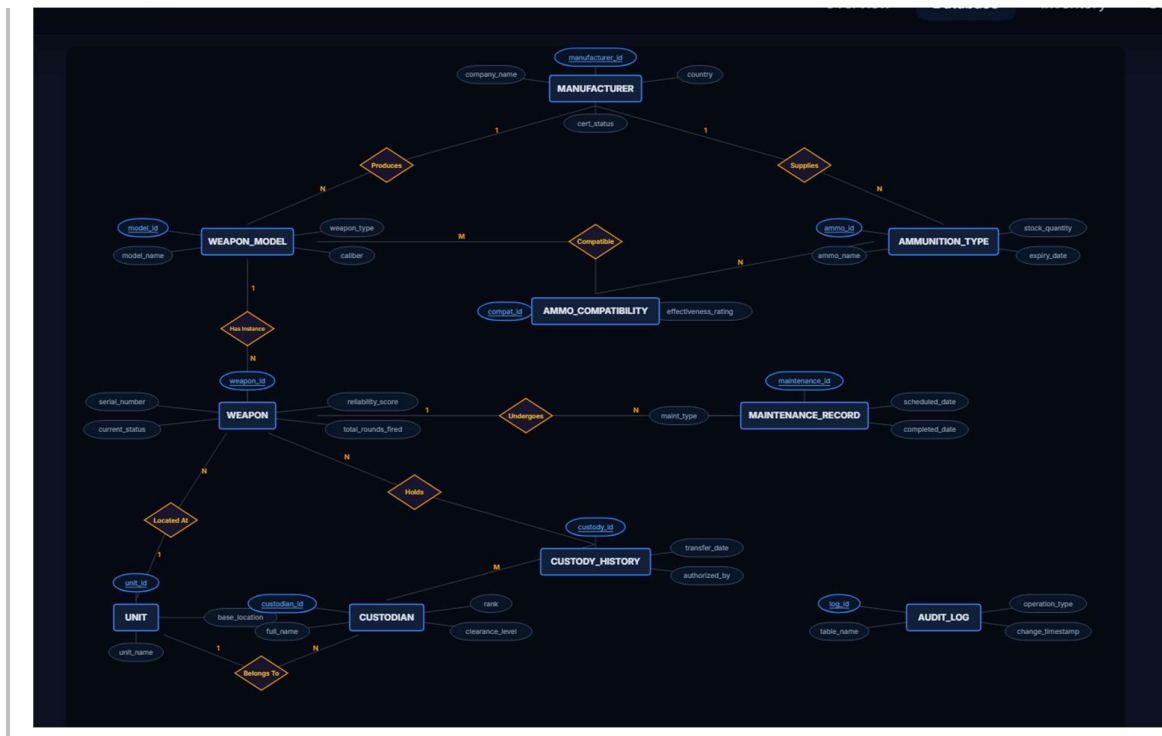


5.2 Key Design Decisions

- **Circular FK handling:** UNIT references CUSTODIAN (commanding officer) and CUSTODIAN references UNIT. Resolved by creating UNIT first without the FK, inserting data, then adding the constraint via ALTER TABLE.
 - **Role-based access via views:** Since Oracle Live SQL does not support CREATE USER/GRANT, RBAC is demonstrated through filtered views.
 - **Sequences starting at 101:** Manual sample data uses IDs 1-100; sequences start at 101 to avoid collisions.
-

6. ER Diagram

6.1 Conceptual ER Diagram (Chen Notation)



The conceptual ER model was designed using classical Chen notation with rectangles for entities, ovals for attributes, diamonds for relationships, and cardinality labels (1:N, M:N). Primary keys are underlined.

6.2 Identified Entities

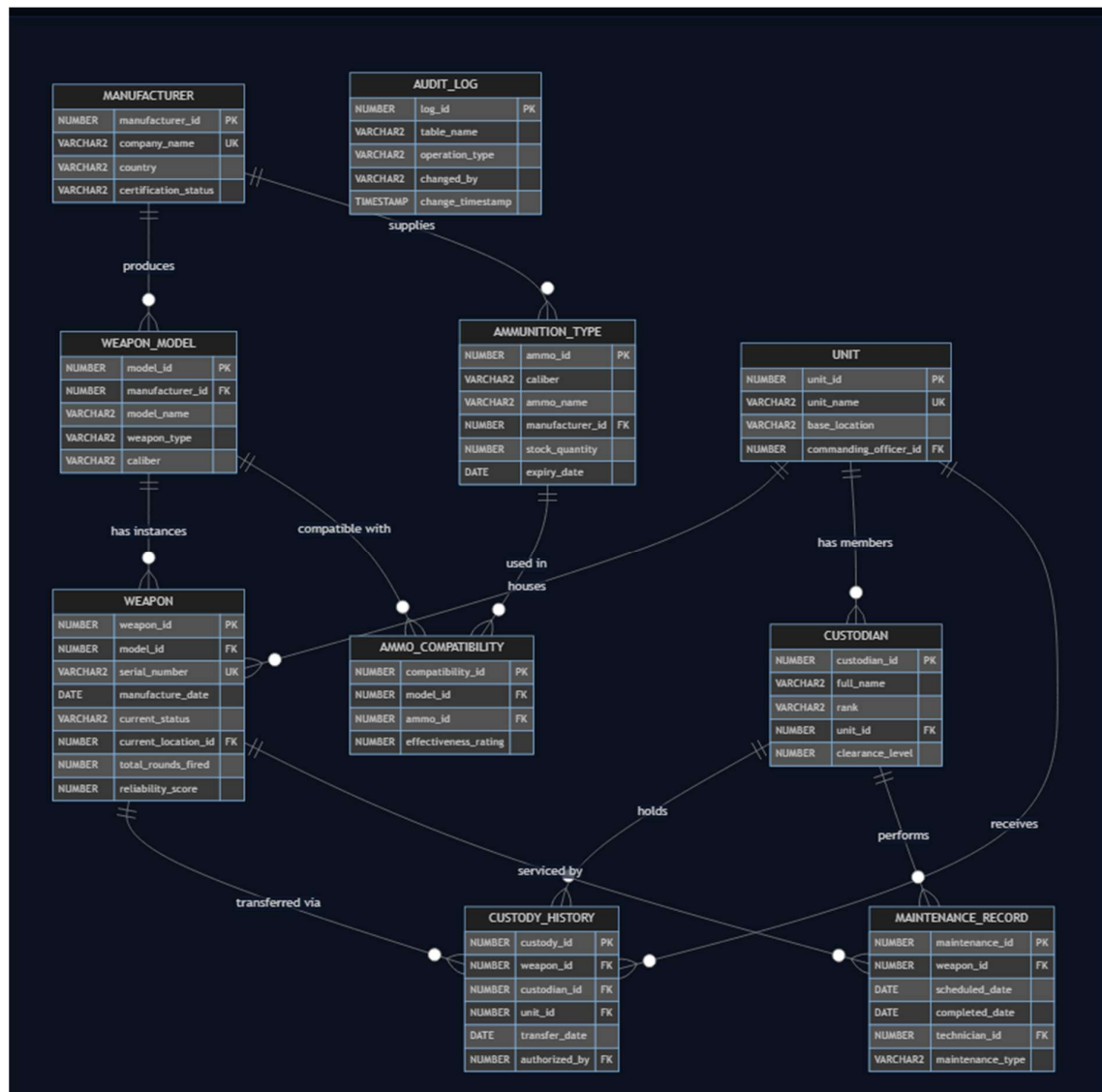
Entity	Description
MANUFACTURER	Stores information about defense equipment manufacturers and their certification status
WEAPON_MODEL	Contains weapon model specifications such as type, caliber, and manufacturer details
WEAPON	Maintains records of individual weapon instances including serial number, status, and location
UNIT	Represents military units along with their base locations and commanding officers
CUSTODIAN	Stores details of personnel responsible for weapon handling and custody
CUSTODY_HISTORY	Tracks weapon transfer history between custodians and units
MAINTENANCE_RECORD	Maintains scheduled and completed weapon maintenance information
AMMUNITION_TYPE	Stores ammunition specifications, stock quantity, and expiry information
AMMO_COMPATIBILITY	Defines compatibility and effectiveness ratings between weapons and ammunition
AUDIT_LOG	Records all important database operations for accountability and auditing purposes

6.3 Key Relationships

Relationship	Type	Description
MANUFACTURER → WEAPON_MODEL	1:N	A manufacturer can produce multiple weapon models
WEAPON_MODEL → WEAPON	1:N	A single weapon model can have many individual weapon instances
UNIT → WEAPON	1:N	A military unit can contain multiple weapons
CUSTODIAN → UNIT	N:1	Multiple custodians can belong to the same unit
WEAPON ↔ CUSTODIAN	M:N	Weapons and custodians share a many-to-many relationship through the CUSTODY_HISTORY table

Relationship	Type	Description
WEAPON_MODEL ↔ AMMUNITION_TYPE	M:N	Weapon models can support multiple ammunition types through the AMMO_COMPATIBILITY table
WEAPON → MAINTENANCE_RECORD	1:N	A weapon can have multiple maintenance and servicing records

6.4 Implementation Relational Schema



7. Relational Schema

7.1 Normalized Tables Summary

#	Table	Primary Key	Foreign Keys	Key Constraints
1	MANUFACTURER	manufacturer_id	—	UNIQUE(company_name), CHECK(certification_status)
2	WEAPON_MODEL	model_id	manufacturer_id → MANUFACTURER	CHECK(weapon_type)
3	UNIT	unit_id	commanding_officer_id → CUSTODIAN	UNIQUE(unit_name)
4	CUSTODIAN	custodian_id	unit_id → UNIT	CHECK(clearance_level BETWEEN 1 AND 5)
5	WEAPON	weapon_id	model_id → WEAPON_MODEL, current_location_id → UNIT	UNIQUE(serial_number), CHECK(current_status, total_rounds_fired, reliability_score)
6	CUSTODY_HISTORY	custody_id	weapon_id → WEAPON, custodian_id → CUSTODIAN, unit_id → UNIT, authorized_by → CUSTODIAN	Maintains transfer history records
7	MAINTENANCE_RECORD	maintenance_id	weapon_id → WEAPON, technician_id → CUSTODIAN	CHECK(maintenance_type, scheduled/completed dates)
8	AMMUNITION_TYPE	ammo_id	manufacturer_id → MANUFACTURER	CHECK(stock_quantity ≥ 0)
9	AMMO_COMPATIBILITY	compatibility_id	model_id → WEAPON_MODEL, ammo_id → AMMUNITION_TYPE	UNIQUE(model_id, ammo_id), CHECK(effectiveness_rating BETWEEN 0 AND 10)
10	AUDIT_LOG	log_id	—	CHECK(operation_type)

Total sample data: 257 rows across 10 tables

8. Normalization

8.1 Functional Dependencies

```
weapon_id → serial_number, model_id, current_status,
total_rounds_fired, reliability_score

model_id → model_name, manufacturer_id, weapon_type, caliber

manufacturer_id → company_name, country, certification_status

(model_id, ammo_id) → effectiveness_rating

custodian_id → full_name, rank, unit_id, clearance_level
```

8.2 First Normal Form (1NF)

All attributes contain atomic values. No repeating groups exist. Each row is uniquely identifiable through primary keys. For example, weapon specifications (manufacturer, caliber, type) are not stored as comma-separated lists but as separate columns with proper data types.

8.3 Second Normal Form (2NF)

No partial dependencies exist. In tables with composite keys (such as AMMO_COMPATIBILITY), the non-key attribute `effectiveness_rating` depends on the full composite key `(model_id, ammo_id)`, not on any subset.

8.4 Third Normal Form (3NF) / BCNF

No transitive dependencies exist. For example:

- Manufacturer details (`company_name`, `country`) are stored in MANUFACTURER, not in WEAPON_MODEL — eliminating the transitive dependency `model_id → manufacturer_id → company_name`.
- Unit details are separated from CUSTODIAN and WEAPON tables.

Every determinant in the schema is a candidate key, satisfying BCNF requirements. The final normalized structure ensures consistency and efficient data maintenance.

9. SQL Implementation

9.1 DDL — Table Creation

Tables were created with comprehensive constraints including PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, and NOT NULL. Indexes were added on frequently queried columns.

```
CREATE TABLE WEAPON (  
    weapon_id          NUMBER          PRIMARY KEY,  
    model_id           NUMBER          NOT NULL,  
    serial_number       VARCHAR2(30)   NOT NULL UNIQUE,  
    manufacture_date    DATE           NOT NULL,  
    current_status      VARCHAR2(20)   NOT NULL  
        CHECK (current_status IN ('Active','In Maintenance',  
        'Decommissioned','In Transit','In Storage')),  
    current_location_id NUMBER          NOT NULL,  
    total_rounds_fired  NUMBER          DEFAULT 0 NOT NULL,  
    reliability_score   NUMBER(5,2)    DEFAULT 100.00,  
    CONSTRAINT fk_weap_model FOREIGN KEY (model_id)  
        REFERENCES WEAPON_MODEL(model_id),  
    CONSTRAINT fk_weap_location FOREIGN KEY (current_location_id)  
        REFERENCES UNIT(unit_id),  
    CONSTRAINT chk_rounds CHECK (total_rounds_fired >= 0),  
    CONSTRAINT chk_reliability CHECK (reliability_score BETWEEN 0  
AND 100)  
);  
  
CREATE INDEX idx_weapon_status ON WEAPON(current_status);  
CREATE INDEX idx_weapon_location ON WEAPON(current_location_id);
```

9.2 DML — Sample Data

257 rows of realistic sample data were inserted across all 10 tables, including 10 manufacturers, 20 weapon models, 12 military units, 30 custodians, 50 weapons, 40 custody transfers, 35 maintenance records, 15 ammunition types, 25 compatibility ratings, and 20 audit log entries.

9.3 Multi-Table JOIN

```
SELECT w.weapon_id, w.serial_number, wm.model_name,  
       wm.weapon_type, m.company_name AS manufacturer,  
       w.current_status, u.unit_name,
```

```

        w.total_rounds_fired, w.reliability_score
FROM WEAPON w
JOIN WEAPON_MODEL wm ON w.model_id = wm.model_id
JOIN MANUFACTURER m ON wm.manufacturer_id = m.manufacturer_id
JOIN UNIT u ON w.current_location_id = u.unit_id
ORDER BY w.weapon_id;

```

9.4 Correlated Subquery Overdue Maintenance

```

SELECT w.weapon_id, w.serial_number, wm.model_name
FROM WEAPON w
JOIN WEAPON_MODEL wm ON w.model_id = wm.model_id
WHERE w.current_status = 'Active'
AND EXISTS (
    SELECT 1 FROM MAINTENANCE_RECORD mr
    WHERE mr.weapon_id = w.weapon_id
        AND mr.next_service_due < SYSDATE
        AND mr.completed_date IS NOT NULL
);

```

9.5 Aggregate Query with HAVING

```

SELECT wm.model_name, wm.weapon_type,
       COUNT(*) AS weapon_count,
       ROUND(AVG(w.reliability_score), 2) AS avg_reliability
FROM WEAPON w
JOIN WEAPON_MODEL wm ON w.model_id = wm.model_id
GROUP BY wm.model_name, wm.weapon_type
HAVING AVG(w.reliability_score) < 90
ORDER BY avg_reliability ASC;

```

9.6 Views for Role-Based Access

Six views were created to implement role-based data access:

View	Target User	Description
VW_ARMORY_OFFICER	Armory Officer	Provides complete inventory details including custody and maintenance information
VW_UNIT_COMMANDER	Unit Commander	Displays weapons assigned only to the commander's unit
VW_MAINTENANCE_TECH	Maintenance Technician	Shows maintenance records along with operational status indicators
VW_AUDITOR	Auditor	Provides read-only access to audit logs and activity history
VW_WEAPON_READINESS	All Authorized Users	Displays weapon readiness classification and operational status dashboard
VW_AMMO_STOCK	Armory Officer	Shows ammunition inventory along with compatibility details for weapon models

```

CREATE OR REPLACE VIEW VW_WEAPON_READINESS AS
SELECT w.weapon_id, w.serial_number, wm.model_name,
       wm.weapon_type, u.unit_name,
       w.current_status, w.reliability_score,
       CASE
         WHEN w.current_status = 'Decommissioned' THEN 'RETIRED'
         WHEN w.current_status IN ('In Maintenance', 'In Transit')
         THEN 'UNAVAILABLE'
         WHEN w.current_status = 'In Storage' THEN 'RESERVE'
         WHEN w.reliability_score >= 95 THEN 'COMBAT READY'
         WHEN w.reliability_score >= 85 THEN 'FIELD READY'
         WHEN w.reliability_score >= 70 THEN 'LIMITED DUTY'
         ELSE 'NEEDS OVERHAUL'
       END AS readiness_class
FROM WEAPON w
JOIN WEAPON_MODEL wm ON w.model_id = wm.model_id
JOIN UNIT u ON w.current_location_id = u.unit_id;

```

10. PL/SQL Implementation

10.1 Stored Procedures

TRANSFER_CUSTODY —” Performs an atomic custody transfer with full validation, SAVEPOINT, and ROLLBACK on failure.

```

CREATE OR REPLACE PROCEDURE TRANSFER_CUSTODY (
    p_weapon_id IN NUMBER, p_new_custodian IN NUMBER,
    p_new_unit IN NUMBER, p_authorized_by IN NUMBER
) AS
    v_weapon_status WEAPON.current_status%TYPE;
    v_auth_clearance CUSTODIAN.clearance_level%TYPE;
BEGIN
    SAVEPOINT before_transfer;

    SELECT current_status INTO v_weapon_status
    FROM WEAPON WHERE weapon_id = p_weapon_id;
    IF v_weapon_status IN ('Decommissioned','In Maintenance') THEN
        RAISE_APPLICATION_ERROR(-20002,
            'Cannot transfer: status ' || v_weapon_status);
    END IF;

    SELECT clearance_level INTO v_auth_clearance
    FROM CUSTODIAN WHERE custodian_id = p_authorized_by;
    IF v_auth_clearance < 4 THEN
        RAISE_APPLICATION_ERROR(-20006, 'Insufficient clearance');
    END IF;

    INSERT INTO CUSTODY_HISTORY VALUES (
        custody_seq.NEXTVAL, p_weapon_id, p_new_custodian,
        p_new_unit, SYSDATE, p_authorized_by, 'Standard transfer');
    UPDATE WEAPON SET current_location_id = p_new_unit,
        current_status = 'Active'
    WHERE weapon_id = p_weapon_id;

    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK TO before_transfer;
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
        RAISE;
END;

```

Other procedures implemented: REGISTER_WEAPON, SCHEDULE_MAINTENANCE, DECOMMISSION_WEAPON, BULK_TRANSFER_UNIT.

10.2 Functions

Function	Return Type	Description
CALC_RELIABILITY_SCORE	NUMBER	Calculates weapon reliability based on rounds fired, maintenance history, and operational age

Function	Return Type	Description
GET_NEXT_SERVICE_DATE	DATE	Determines the next scheduled maintenance date for a weapon
CHECK_AMMO_COMPATIBILITY	VARCHAR2	Validates ammunition compatibility with weapon models and checks ammunition expiry
GET_WEAPON_STATUS_SUMMARY	VARCHAR2	Returns a formatted summary of weapon status and operational details
COUNT_ACTIVE_WEAPONS	NUMBER	Counts the number of active weapons assigned to a specific unit
GET_CUSTODY_DURATION	NUMBER	Calculates the number of days a custodian has possessed a weapon

```

CREATE OR REPLACE FUNCTION CALC_RELIABILITY_SCORE (
    p_weapon_id IN NUMBER
) RETURN NUMBER AS
    v_rounds NUMBER; v_mfg_date DATE;
    v_age NUMBER; v_maint NUMBER; v_overdue NUMBER;
    v_score NUMBER := 100;
BEGIN
    SELECT total_rounds_fired, manufacture_date
    INTO v_rounds, v_mfg_date
    FROM WEAPON WHERE weapon_id = p_weapon_id;

    v_age := MONTHS_BETWEEN(SYSDATE, v_mfg_date) / 12;
    SELECT COUNT(*) INTO v_maint FROM MAINTENANCE_RECORD
    WHERE weapon_id = p_weapon_id AND completed_date IS NOT NULL;
    SELECT COUNT(*) INTO v_overdue FROM MAINTENANCE_RECORD
    WHERE weapon_id = p_weapon_id
        AND completed_date IS NULL AND scheduled_date < SYSDATE;

    v_score := v_score - FLOOR(v_rounds / 5000)
        - (v_age * 0.5) - (v_overdue * 5)
        + LEAST(v_maint, 10);
    RETURN GREATEST(LEAST(ROUND(v_score, 2), 100), 0);
END;

```

10.3 Triggers

Trigger	Event	Description
TRG_AUDIT_WEAPON	AFTER INSERT/UPDATE/DELETE ON WEAPON	Automatically logs all weapon-related changes into the AUDIT_LOG table
TRG_AUDIT_CUSTODY	AFTER INSERT ON CUSTODY_HISTORY	Records all custody transfer operations for accountability and traceability
TRG_MAINTENANCE_ALERT	AFTER INSERT/UPDATE ON MAINTENANCE_RECORD	Generates alerts for overdue or pending maintenance activities
TRG_UPDATE_RELIABILITY	BEFORE UPDATE OF total_rounds_fired ON WEAPON	Automatically recalculates weapon reliability scores after usage updates
TRG_CHECK_AMMO_EXPIRY	BEFORE INSERT ON AMMO_COMPATIBILITY	Prevents assignment of expired ammunition to weapon models
TRG_WEAPON_STATUS_GUARD	BEFORE UPDATE OF current_status ON WEAPON	Prevents decommissioned weapons from being reactivated accidentally

```
CREATE OR REPLACE TRIGGER TRG_AUDIT_WEAPON
AFTER INSERT OR UPDATE OR DELETE ON WEAPON
FOR EACH ROW
DECLARE
    v_log_id NUMBER; v_op VARCHAR2(10);
BEGIN
    SELECT audit_log_seq.NEXTVAL INTO v_log_id FROM DUAL;
    IF INSERTING THEN v_op := 'INSERT';
    ELSIF UPDATING THEN v_op := 'UPDATE';
    ELSE v_op := 'DELETE'; END IF;

    INSERT INTO AUDIT_LOG (log_id, table_name, operation_type,
        record_id, new_values, changed_by)
    VALUES (v_log_id, 'WEAPON', v_op,
        NVL(:NEW.weapon_id, :OLD.weapon_id),
        'Status=' || NVL(:NEW.current_status, :OLD.current_status),
        USER);
END;
```


10.4 Cursors

Six cursor types were implemented:

1. **Explicit Cursor** ” Overdue maintenance report using OPEN/FETCH/CLOSE
2. **Parameterized Cursor** ” Unit weapons inventory with parameter binding
3. **Cursor FOR Loop** ” Fleet reliability report across all active weapons
4. **REF Cursor** ” Dynamic query accepting status parameter at runtime
5. **Cursor FOR UPDATE** ” Batch reliability recalculation with row locking
6. **BULK COLLECT** ” Expired ammunition processing using PL/SQL collections

11. Transaction Management & Concurrency

11.1 ACID Properties Demonstrated

ACID Property	Implementation
Atomicity	Custody transfer operations combine INSERT and UPDATE statements into a single transaction unit. If any operation fails, <code>ROLLBACK TO SAVEPOINT</code> restores the previous state.
Consistency	CHECK constraints, foreign key constraints, and triggers ensure that all database records remain valid and logically consistent.
Isolation	<code>SELECT ... FOR UPDATE</code> is used to lock weapon records during transfers, preventing concurrent modifications from other transactions.
Durability	Once a <code>COMMIT</code> operation is executed, all changes are permanently stored using Oracle’s transaction logging mechanism.

11.2 SAVEPOINT and Partial ROLLBACK

```
SAVEPOINT step1_maintenance;  
INSERT INTO MAINTENANCE_RECORD VALUES (...); -- Step 1: OK  
  
SAVEPOINT step2_status;  
UPDATE WEAPON SET current_status = 'Active' WHERE weapon_id = 2; -
```

```

- Step 2: OK

BEGIN
    UPDATE WEAPON SET total_rounds_fired = -100 WHERE weapon_id =
2; -- Step 3: FAILS
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK TO step2_status; -- Only step 3 rolled back
END;

COMMIT; -- Steps 1 & 2 persisted

```

11.3 Row-Level Locking

```

SELECT current_status INTO v_status
FROM WEAPON WHERE weapon_id = 5
FOR UPDATE; -- Row locked " other sessions must wait

-- Perform transfer while row is locked
INSERT INTO CUSTODY_HISTORY VALUES (...);
COMMIT; -- Lock released

```

12. Frontend Dashboard

A single-page HTML/CSS/JavaScript website was created to serve as a project showcase dashboard. The website requires no build tools or server " it runs directly by opening `index.html` in a browser.

12.1 Technology Stack

- **HTML5** " Semantic page structure
- **CSS3** " Dark military-tech theme with glassmorphism effects
- **JavaScript** " Interactive components and data rendering
- **Chart.js** " Analytics visualizations
- **Mermaid.js** " ER diagram rendering
- **Prism.js** " SQL syntax highlighting

12.2 Dashboard Sections

Section	Description
Hero / Overview	Displays the project title, key statistics (10 tables, 257 rows, 23 PL/SQL objects), and technologies used
Database Design	Presents the conceptual ER diagram using Chen notation along with the relational schema using Crow's Foot notation
Weapon Inventory	Provides a searchable, sortable, and filterable table containing all 50 weapon records with status indicators
SQL/PL-SQL Showcase	Includes a tab-based interface for displaying SQL and PL/SQL scripts with syntax highlighting
Analytics Dashboard	Visualizes system analytics through charts such as weapon status distribution, weapons per unit, reliability by weapon type, and ammunition stock levels

12.3 Deployment

The website is deployed via GitHub Pages, providing a live URL for evaluators to access without any setup.

13. Challenges Faced

Challenge	Resolution
Oracle Live SQL session timeouts	Large insertion scripts were divided into smaller batches, and progress was saved frequently to avoid data loss
Circular foreign key dependency (UNIT ↔ CUSTODIAN)	The UNIT table was initially created without the foreign key constraint, data was inserted into both tables, and the constraint was later added using <code>ALTER TABLE</code>
Trigger debugging	<code>DBMS_OUTPUT.PUT_LINE</code> statements were used extensively for tracing execution flow, and triggers were tested individually before integration
Foreign key insertion ordering	INSERT statements were carefully sequenced to maintain referential integrity by inserting parent records before child records

Challenge	Resolution
Maintaining normalization	Functional dependencies were continuously reviewed during schema modifications to prevent transitive dependencies and redundancy
Oracle Live SQL paste limitations	Large data insertion scripts were separated into multiple files (03a, 03b, 03c) for easier execution
Cursor mutating table errors	BEFORE triggers were used instead of AFTER triggers when modifications involved the same table row being processed

14. Future Scope

1. **User Authentication:** Implement proper login system with Oracle database users and GRANT-based privileges instead of view-based access control.
 2. **Cloud Deployment:** Deploy the Oracle database on Oracle Cloud Free Tier for persistent access and multi-user testing.
 3. **Real-Time Monitoring:** Add a live dashboard that connects to the database via REST APIs for real-time inventory tracking.
 4. **RFID/Barcode Integration:** Integrate barcode scanning for automated asset check-in/check-out at armory gates.
 5. **Mobile Application:** Develop a mobile app for field officers to view weapon status and submit maintenance requests.
 6. **Advanced Analytics:** Implement predictive maintenance models using historical reliability and usage data.
-

15. Conclusion

This project successfully demonstrates the design and implementation of a relational database system for defense asset management using Oracle SQL and PL/SQL. The key outcomes include:

- A **normalized relational database** (3NF/BCNF) with 10 tables, 257 rows of sample data, and comprehensive integrity constraints.
- **16 SQL queries** demonstrating joins, subqueries, aggregates, views, and complex filtering.

- **23 PL/SQL objects** (5 procedures, 6 functions, 6 triggers, 6 cursor blocks) automating custody transfers, maintenance scheduling, reliability calculations, and audit logging.
- **ACID-compliant transaction management** with COMMIT, ROLLBACK, SAVEPOINT, and row-level locking.
- **A frontend dashboard** providing interactive visualization of the database contents.

The project reinforced practical understanding of core DBMS concepts including ER modeling, normalization, referential integrity, PL/SQL programming, and transaction control applied to a realistic problem domain with clear societal relevance.

References

1. Elmasri, R. & Navathe, S.B.” *Fundamentals of Database Systems* (7th Edition)
2. Oracle Documentation” *PL/SQL Language Reference*
3. Oracle Documentation” *SQL Language Reference*
4. Oracle Live SQL” <https://livesql.oracle.com>
5. United Nations SDG-16” <https://sdgs.un.org/goals/goal16>

End of Report